

Reviewer Pack — Minimal Rank of Quandle Representations Bounds Monotone Circ...

Entry 5545abd6660a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 15:18:34 UTC

Minimal Rank of Quandle Representations Bounds Monotone Circuit Size

Entry ID: 5545abd6660a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 15:18:34 UTC

1. Conjecture

Field A (mathematical branch): Quandle Theory **Field B** (complexity object): Complexity Theory: Boolean Function Complexity

Statement:

{‘text’: ‘For any quandle Q , there exists a constant c_Q such that the minimal rank of the representation of any monotone boolean function f represented by Q is at least $c_Q * \log(n)$, where n is the number of variables in f .’, ‘counterexample’: ‘A counterexample would be a quandle representation for a monotone boolean function with a minimal rank less than $c_Q * \log(n)$ for all n .’}

Rationale (proposer’s reasoning):

{‘text’: ‘Quandle Theory provides algebraic structures that can encode combinatorial properties, potentially leading to a deeper understanding of circuit complexity. If the conjecture holds, it suggests that quandle representations could be a powerful tool in bounding monotone circuit size, offering new insights into the nature of computational complexity.’}

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 422cbbd74559f495

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across 30 random seeds, for all $n \leq 40$, the minimal rank of quandle representations for monotone boolean functions satisfies ‘ $\min_rank \geq c_Q * \log(n)$ ’. The conjecture is falsified if there exists any seed where ‘ $\min_rank < c_Q * \log(n)$ ’ for some $n \leq 40$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "quandle theory" AND "boolean function complexity" AND minimal rank" - "monotone circuit size" in quandles AND lower bound on representation" - "c_Q * log(n)" as a threshold in quandle representations for monotone functions

Top relevant hits considered: - [http://arxiv.org/abs/2605.12692v1] On quandle representations - [http://arxiv.org/abs/2307.03728v1] On the representation theory of cyclic and dihedral quandles - [http://arxiv.org/abs/2604.12550v1] On irreducible representations of quandles - [http://arxiv.org/abs/1411.4413v2] Observation of the rare $B_s^0 \rightarrow \mu^+ \mu^-$ decay from the combined analysis of CMS and LHCb data - [http://arxiv.org/abs/0901.0512v4] Expected Performance of the ATLAS Experiment - Detector, Trigger and Physics - [http://arxiv.org/abs/2601.07595v3] Deep Search for Joint Sources of Gravitational Waves and High-Energy Neutrinos with IceCube During the Third Observing R

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=14.9s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def is_monotone(f):
        n = len(f)
        for i in range(n):
            for j in range(i + 1, n):
                if f[i] > f[j]:
                    return False
        return True

    def generate_random_monotone_function(n):
        inputs = [random.choice([0, 1]) for _ in range(n)]
        outputs = sorted(inputs)
        return outputs

    def compute_minimal_rank(f):
```

```

n = len(f)
identity_matrix = [[Fraction(1 if i == j else 0) for j in range(n)] for i in range(n)]

def matrix_multiply(A, B):
    result = [[sum(a * b for a, b in zip(row_A, col_B)) for col_B in zip(*B)] for row_A in A]
    return result

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        max_row = max(range(i, n), key=lambda k: abs(matrix[k][i]))
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        denom = matrix[i][i]
        if denom == 0:
            continue

        for j in range(n):
            matrix[i][j] /= denom

        for k in range(n):
            if k != i:
                factor = matrix[k][i]
                for j in range(n):
                    matrix[k][j] -= factor * matrix[i][j]

    return matrix

reduced_matrix = gaussian_elimination(identity_matrix)

rank = sum(1 for row in reduced_matrix if any(row))
return rank

results = []

for n in [5, 10, 15, 20, 30, 40]:
    for _ in range(5): # Ensure at least 30 instances per seed
        f = generate_random_monotone_function(n)
        if not is_monotone(f):
            return {
                "metric_name": "minimal_rank",
                "metric_value": None,
                "instances_tested": 1,
                "conjecture_holds": False,
                "counterexample": "Non-monotone function generated"
            }

    min_rank = compute_minimal_rank(f)
    results.append((n, min_rank))

c_Q = sum(min_rank for n, min_rank in results) / len(results)
mean_rank = sum(min_rank * (math.log(n) if n > 0 else 1) for n, min_rank in results) / sum(mat

conjecture_holds = all(min_rank >= c_Q * math.log(n) for n, min_rank in results)

return {

```

```

        "metric_name": "minimal_rank",
        "metric_value": mean_rank,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else "c_Q * log(n) bound violated"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **trial_result}}")
        results.append(trial_result)

    mean_rank = sum(result["metric_value"] for result in results if result["metric_value"] is not None) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all("counterexample" in result and result["counterexample"] == "c_Q * log(n) bound violated" for result in results):
        print(f"RESULT: FALSIFIED counterexample=\"c_Q * log(n) bound violated\" first_failing_seed={seeds[0]}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0 support_fraction={support_fraction}")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

TRIAL: {"seed": 11, **trial_result}
TRIAL: {"seed": 23, **trial_result}
TRIAL: {"seed": 37, **trial_result}
TRIAL: {"seed": 53, **trial_result}
TRIAL: {"seed": 71, **trial_result}
TRIAL: {"seed": 89, **trial_result}
TRIAL: {"seed": 103, **trial_result}
TRIAL: {"seed": 127, **trial_result}
TRIAL: {"seed": 149, **trial_result}
TRIAL: {"seed": 167, **trial_result}
TRIAL: {"seed": 191, **trial_result}
TRIAL: {"seed": 211, **trial_result}
TRIAL: {"seed": 233, **trial_result}
TRIAL: {"seed": 257, **trial_result}
TRIAL: {"seed": 277, **trial_result}
TRIAL: {"seed": 311, **trial_result}
TRIAL: {"seed": 347, **trial_result}
TRIAL: {"seed": 389, **trial_result}
TRIAL: {"seed": 421, **trial_result}
TRIAL: {"seed": 463, **trial_result}
TRIAL: {"seed": 503, **trial_result}
TRIAL: {"seed": 547, **trial_result}
TRIAL: {"seed": 593, **trial_result}

```

```

TRIAL: {"seed": 631, **trial_result}
TRIAL: {"seed": 677, **trial_result}
TRIAL: {"seed": 727, **trial_result}
TRIAL: {"seed": 773, **trial_result}
TRIAL: {"seed": 821, **trial_result}
TRIAL: {"seed": 877, **trial_result}
TRIAL: {"seed": 929, **trial_result}
RESULT: FALSIFIED counterexample="c_Q * log(n) bound violated" first_failing_seed=11

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test has only considered up to 15 instances ($n \leq 15$), which may not be sufficient to establish a general trend. The conjecture's validity could depend on the behavior of larger n , and the current results might simply reflect a trivial sub-case or an artifact of the tested range.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test has identified a counterexample where the minimal rank of quandle representations for monotone boolean functions does not satisfy the conject | next: Further investigation is needed to determine if this counterexample holds for larger values of n or if it is an artifact of the tested range. Additional tests with a wider range of n and more seeds are recommended.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14711
2	preregistration	ollama_remote	glm4:latest	0	0	5695
3	novelty	ollama_remote	glm4:latest	0	0	4681
4	novelty	ollama_remote	glm4:latest	0	0	6175
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13194
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	37997
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19198
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12861
9	critic	ollama_remote	glm4:latest	0	0	51301
10	judge	ollama_remote	glm4:latest	0	0	7345

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 173157 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in [research/audit/5545abd6660a.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5545abd6660a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/5545abd6660a.tar.gz` (if generated)